Uncovering LLM Lineage via Static Fingerprints: Progress, Gaps, and Opportunities

Anonymous Author(s)

Abstract

CCS Concepts

• **Do Not Use This Code → Generate the Correct Terms for Your Paper**; *Generate the Correct Terms for Your Paper*; Generate the Correct Terms for Your Paper; Generate the Correct Terms for Your Paper.

Keywords

Do, Not, Use, This, Code, Put, the, Correct, Terms, for, Your, Paper

ACM Reference Format:
Anonymous Author(s). 2026. Uncovering LLM Lineage via Static Fingerprints: Progress, Gaps, and Opportunities. In . ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Large Language Models (LLMs) have significantly transformed the landscape of artificial intelligence and software engineering. While proprietary models such as GPT and Claude dominate the consumer market, public open-source models such as DeepSeek are increasingly gaining popularity among business entities due to their enhanced transparency, lower deployment costs, reduced legal and licensing constraints, and greater flexibility for customization. Indeed, popular model-sharing platforms (e.g., Hugging Face) host over 2 million models [2], with top models receiving more than one million downloads [45].

Notably, a closer examination of the interconnections among these models reveals a significant degree of reuse [45]. In particular, many models are derived from existing base models through techniques such as fine-tuning [22], model merging [6], and quantization [94]. Such derivations enable the adaptation of general-purpose models to domain-specific scenarios, capability enhancement, and efficiency optimization. However, while enjoying the benefits, the model reusing also bring the inherited risks [13, 18, 32, 79], including legal liabilities (e.g., Intellectual Property violations) to security threats [25, 42, 44] (e.g., backdoors, jailbreaks, and prompt injection).

An effective strategy for risk mitigation is to ensure transparency in model composition, such as using AI Bills of Materials (AI-BOM) [1]. However, although platforms provide mechanisms (e.g., model cards in Hugging Face [5]) that allow contributors to document meta-information, such as model reuse details or model performance [60], such practices are not consistently adopted proactively by model developers. This leads to incomplete disclosure of model information. For instance, prior work reports that the ninth most downloaded pretrained model on Hugging Face, with over 6.9 million monthly downloads, lacks a model card [45]. Consequently, it undermines model transparency, thereby hindering effective risk traceability and mitigation.

To address this problem, a range of model fingerprinting techniques can be leveraged. While dynamic fingerprinting approaches rely on analyzing a model's inference behavior in response to carefully designed input queries [26, 27, 63, 65, 66, 87–91, 95, 107], they are often sensitive to input selection and may suffer from limited robustness under model modifications such as fine-tuning or compression. Therefore, we focus on the problem of *model lineage construction via static fingerprints*.

Empirical Study. We begin with an empirical study to understand the dependency prevalence and dependency types of public model reuse.

The problem of opaque model lineage is increasingly pressing. LLMs are often trained on massive datasets collected from multiple sources, combined with pre-trained models, and further fine-tuned in proprietary pipelines. This complexity obscures the relationship between models, making it difficult to trace the origin of errors, detect unintended biases, or comply with licensing requirements. Consequently, software engineers, AI practitioners, and policymakers face challenges in maintaining trust, reliability, and accountability in AI-driven systems. Despite the growing literature on supply chain analysis in traditional software and deep learning frameworks, systematic studies focused on LLM lineage remain sparse.

This paper seeks to address this gap by investigating how far current methods go in tracing and analyzing the lineage of LLMs. Specifically, we ask: (1) What techniques exist to track dependencies and derivations among LLMs? (2) How effective are these techniques in providing visibility into model origins and transformations? (3) What limitations persist, and where is further research needed? By framing LLM lineage in the context of software engineering, we aim to connect advances in AI with established practices in software maintenance, traceability, and quality assurance.

2 Motivation

The proliferation of Large Language Models (LLMs) has been dramatically accelerated by collaborative platforms like the Hugging Face Hub, which serves as a central repository for model sharing. A cornerstone of this ecosystem is the iterative development of new models by building upon existing ones through techniques such as fine-tuning, merging, and quantization. This process creates a complex web of inter-model dependencies, which we define as **model genealogy** or **lineage**. A clear understanding of a model's genealogy is paramount for ensuring traceability, reproducibility,

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, July 2017, Washington, DC, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

and compliance, as derivative models often inherit biases, vulnerabilities, or licensing constraints from their predecessors.

However, the mechanisms currently available on the Hugging Face Hub for documenting this critical lineage information are frequently inconsistent, incomplete, or entirely absent. This lack of a systematic and enforceable standard for provenance documentation poses a significant barrier to researchers and developers. The most visible symptom of this issue is the difficulty, or outright impossibility, of programmatically constructing a model's lineage–a structure ideally represented as a "Model Tree". This is not a superficial interface problem but a direct consequence of fundamental flaws in how provenance data is recorded at its source.

To illustrate the severity of this challenge, we will dissect two distinct modes of documentation failure, each representing a progressively worsening degree of information obfuscation.

2.1 Problem 1: Lineage Concealed in Unstructured Text

In the most common scenario, lineage information exists but is locked within free-form text, rendering it inaccessible to automated tools. This forces researchers into a manual, error-prone process of discovery.

Case Study 1: `DavidAU/NSFW_DPO_Noromaid-7b-GGUF`

The Hugging Face repository of this model does not include a model tree; however, its model card explicitly states:

> *NSFW_DPO_Noromaid-7b-Mistral-7B-Instruct-v0.1 is a merge of the following models: mistralai/Mistral-7B-Instruct-v0.1 and athirdpath/NSFW_DPO_Noromaid-7b.*

Notably, the Hugging Face pages of both referenced models in the model card contain their respective model trees. This reliance on unstructured documentation makes the automated construction of a lineage graph infeasible. It imposes a significant manual burden for identifying even a direct parent and completely obstructs any large-scale, systematic analysis of the model ecosystem.

Case Study 2: `timdettmers/guanaco-13b`

Another example is the `timdettmers/guanaco-13b` model. While this model is a fine-tuned version of LLaMA, this pivotal relationship is not captured in any structured, machine-readable metadata. To discover this link, a user must manually navigate to the model's repository, open the "Model Card" (a Markdown file), and parse through natural language prose to find the statement:

> *Guanaco models are open-source finetuned chatbots obtained through 4-bit QLoRA tuning of LLaMA base models...*

2.2 Problem 2: Lineage Outsourced to Fragile External Links

A more precarious situation arises when provenance information is not even located within the Hugging Face platform, but is instead hyperlinked to an external, third-party source.

Case Study 3: `pfnet/plamo-2.1-2b-cpt`

The model's homepage lacks a model tree, and its model card merely states that the model was derived by reducing the parameter size from an 8B-parameter model, without explicitly specifying the base model used for this operation. However, the model card provides links to three technical blogs for reference. By following these external links, we can infer that the mentioned 8B-parameter model is `plano-2-8b`.

Case Study 4: `Remilistrasza/Mysterious_SDXL_Lah`

Consider the `Remilistrasza/Mysterious_SDXL_Lah` model. Its Model Card is starkly empty of descriptive content, offering only a single hyperlink to a page on Civitai, another model-sharing platform. To identify the base model, a user is forced to leave the Hugging Face ecosystem. Only upon navigating to this external page does one discover that the base model is "*SDXL 1.0*".

This practice introduces two severe risks: **information fragmentation** and **link rot**. The integrity and availability of the lineage information become wholly dependent on the uptime and archival policies of an external website. Should the link become invalid (a 404 error), or the content on the remote page be altered or deleted, the model's traceable lineage is permanently severed. This practice renders the model's provenance fragile and unverifiable from within the Hugging Face ecosystem.

These documented failures, which range from information concealed in prose to its complete omission, demonstrate a clear and pressing need for an automated and reliable method to construct model genealogies. The current reliance on inconsistent and manual documentation is an unsustainable bottleneck in an ecosystem that is scaling rapidly. To bridge this critical gap, we propose an approach to automatically identify and verify the derivative relationships between models, thereby enabling robust and scalable lineage tracing.

2.3 Literature on Model Lineage Construction

As the problem of model traceability gains prominence, various methods have been proposed to detect relationships between LLMs (summarized in Table 1). However, these approaches typically over-optimize for specific operational scenarios, severely limiting their generalizability when constructing comprehensive lineage trees in the wild. First, the majority of techniques (e.g., REEF [107], MDIR [108], HuRef [105]) treat provenance as a symmetrical, pairwise similarity problem (Type "P") rather than a directional graph, failing to identify which model is the parent and which is the child (Dir. "×"). Second, they suffer from strict evaluation biases: dynamic

Name	Year	Acc.	Analy.	Type	Dir.	Evolution Support
TensorGuard [88]	2025	W	D	P	×	FT, PT
LLM DNA [87]	2026	B	D	L	×	FT, PT, D
Model Map [63]	2025	W	D	L	✓	FT, M, Q, D
REEF [107]	2025	W	D	P	×	FT, M, P, D
MDIR [108]	2025	W	S	P	×	FT, P, UC
PhyloLM [98]	2025	B	D	L	×	FT, Q
Mother [30]	2025	W	S	L	✓	FT, PT
Model Atlas [29]	2025	W	S	L	✓	FT, M, Q, D
Neural Lineage [101]	2024	W	S, D	P	×	FT
HuRef [105]	2024	W	S	P	×	FT, PT
SeedPrints [76]	2026	W	D	P	×	FT, M, PT, Q, D
PDF [100]	2025	W	S	P	×	FT, UC

Table 1: Acc.(ess): Black/White-box; Analy.(sis): Static, Dynamic; Type: Pair-wise, Lineage-tree; Dir.: Directional inference; Evolution: FT: Fine-tuning, M: Merging, PT: PEFT, Q: Quantization, P: Pruning, UC: Upcycling, D: Distillation.

methods rely on specific test prompts that are vulnerable to output homogeneity [63, 87, 88, 98, 107], while static weight-based methods often require rigid architectural alignment [100, 105, 108]. Consequently, while highly accurate for standard fine-tuning (FT), most methods fail to generalize across the complex, combinatorial operations actually prevalent, such as model merging (M), quantization (Q), Distillation (D) and pruning (P). Although a few recent works like Mother [30] and Model Atlas [29] attempt directional lineage tracing, they still suffer in real-world scenarios due to limited evolution support and fragile directional inference: Mother is strictly confined to basic fine-tuning, while Model Atlas relies on intra-architecture weight distances that completely fail to capture cross-architecture evolutions (e.g., distillation) or subtle alignments (e.g., RLHF). This highlights the urgent need for a more generalized, operation-agnostic framework for automated genealogy construction.

2.4 A Preliminary Investigation on Hugging Face Model Card

[29]

3 Benchmark

3.1 Construction

The construction of our benchmark involves a systematic pipeline that traces model derivations from base models to their descendants on Hugging Face. This process is divided into five key steps:

Step 1: Obtaining Base Model List. We begin by collecting a list of popular base models for the `text-generation` task from Hugging Face. Utilizing the `HfApi` provided by `huggingface_hub`, we retrieve the top 50 non-gated models sorted by the number of likes. The model identifiers are stored in a file named `base_models.txt`, where each line corresponds to one base model ID. This step is implemented via **Script 1**.

Step 2: Building the Model Derivation Tree. Based on `base_models.txt`, we recursively crawl derivative models to construct a "model forest". For each base model, we explore its derivatives up to four levels deep, covering four derivation types: `finetune`, `adapter`, `quantized`, and `merge`. For each derivation type, a maximum of six sibling nodes are considered. Every node in the tree records metadata such as model name, crawl timestamp, depth level, and its child nodes. The resulting forest structure is serialized into `model_forest.json`. This step is implemented via **Script 2**.

Step 3: Constructing Model Pairs. From the model forest, we extract two categories of model pairs: adjacent parent-child pairs and non-adjacent ancestor-descendant pairs. By performing a Breadth-First Search (BFS) traversal on each tree, we generate:

- **Adjacent Pairs**: Direct parent-child model pairs.
- **Non-Adjacent Pairs**: All ancestor-descendant pairs where the ancestor is not the immediate parent.

These pairs are recorded in `adjacent_pairs.jsonl` and `non_adjacent_pairs.jsonl`, respectively. This step is implemented via **Script 3**.

Step 4: Single-Threaded Model Validation. To ensure data quality, we validate the structural correctness of all adjacent model pairs. For each model, we inspect its file tree on Hugging Face, checking for the presence of critical files such as `config.json`, `adapter_config.json`, and various parameter files (e.g., `.bin`, `.pt`, `.safetensors`, etc.). We verify whether the claimed derivation type aligns with the actual file structure and whether the parent model is valid. The results are compiled into a comprehensive validation report named `model_validation_report.json`. This step is implemented via **Script 4**.

Step 5: Filtering Erroneous Model Pairs. Based on the validation report, we remove all erroneous model pairs from the dataset. Models flagged with any inconsistencies or missing critical files are excluded. The cleaned set of adjacent model pairs is saved in `adjacent_pairs_without_error.json`. This step is implemented via **Script 5**.

3.2 Characteristics

Our benchmark exhibits several notable characteristics that reflect its diversity and representativeness of the Hugging Face model ecosystem:

- **Variety of Derivation Types**: The dataset covers four major derivation types—`finetune`, `adapter`, `quantized`, and `merge`—capturing a wide range of model transformation patterns.
- **Hierarchical Depth**: With a recursive crawling depth of up to four levels, the benchmark contains both shallow and deep derivation chains, enabling analysis of both immediate and long-range model inheritance.
- **Scale and Coverage**: Starting from 50 popular base models, the recursive traversal results in a substantial model forest, ensuring coverage of highly influential models as well as their derivative variants.
- **Data Cleanliness**: Through rigorous validation and error filtering, the dataset ensures high structural integrity, making it suitable for tasks that require reliable model lineage information.
- **Real-World Relevance**: The benchmark reflects actual user-driven derivation patterns on Hugging Face, providing insights into community practices such as model fine-tuning trends, adapter usage, and quantization adoption.

Table 2: Statistics of the GrandMoTher benchmark.

Statistic	Value
#Graphs	31
#Model pairs (edges)	248
Min / Median / Max pairs per graph	1 / 6 / 22
Avg. pairs per graph	8.0
#Models (nodes)	279
#Root models	31
#Intermediate models	44
#Leaf models	204
Share of leaf models	73.1%
Min / Avg / Max models per graph	2 / 9.0 / 23
Avg. out-degree of non-leaf models	3.31
Max fine-tuning depth	2

4 Empirical Setup

We design experiments to explore the following research questions.

- **Measurement Evaluation (RQ1)**: How do different measurement metrics affect the effectiveness of LLM lineage detection?
- **Layer Contribution Analysis (RQ2)**: How do different model layers influence the effectiveness of LLM lineage detection?
- **Failure Analysis (RQ3)**: Why do existing methods fail to detect lineage?
- **Integration (RQ4)**: using the above findings, the best performance can achieve? (80%)

Measurement Metrics. We select common measurement metrics that are used in static white-box fingerprinting. Static white-box fingerprinting captures intrinsic weight characteristics, including statistical distributions [100] (e.g., L_2 distances of model tensors [30]), directionally invariant terms [105], SVD-based spectral signatures [78], and matrix-driven homology proofs [108].

Accuracy. To comprehensively evaluate the structural recovery of model lineages, we measure performance at both the edge level and the tree level. Let $\mathcal{T}$ denote the set of all model family trees in our dataset. We define the following two metrics:

- *Micro-accuracy*: This metric evaluates the local edge prediction performance. It is defined as the total number of correctly predicted directed edges divided by the total number of ground-truth edges across all evaluated model graphs:

$$Micro\text{-}acc = \frac{\sum_{T \in \mathcal{T}} |E^{(T)}_{correct}|}{\sum_{T \in \mathcal{T}} |E^{(T)}_{total}|}$$

- *Macro-accuracy*: This metric evaluates the global topology reconstruction. It is defined as the ratio of successfully recovered model family trees (where the entire lineage topology and edge directions are perfectly matched) to the total number of model families:

$$Macro\text{-}acc = \frac{|\{T \in \mathcal{T} \mid T_{recovered} = T_{true}\}|}{|\mathcal{T}|}$$

Layers. To address RQ2, we systematically isolate the core structural components within Transformer-based Large Language Models (LLMs). Specifically, we dissect the Multi-Head Attention mechanism and the Feed-Forward Network (FFN). The attention mechanism involves projection weight matrices for the Query (W_Q), Key (W_K), Value (W_V), and Output (W_O):

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Furthermore, we evaluate the contribution of the FFN, which typically consists of Multi-Layer Perceptron (MLP) dense layers:

$$FFN(x) = \sigma(xW_1 + b_1)W_2 + b_2$$

where σ represents a non-linear activation function. This granular breakdown enables us to systematically evaluate the fingerprinting capacity of various layer combinations and quantify their respective impacts on the effectiveness of lineage detection.

Graph Algorithms. In our experiments, we evaluate and compare several graph construction algorithms to reconstruct the model lineage topology from the computed pairwise distance and directional matrices:

- *MoTher*: Following Horwitz et al. [30], we formulate the base lineage recovery task as finding a minimum directed spanning tree (MDST). We employ the Chu-Liu-Edmonds algorithm [14, 17] to achieve this optimal cycle-free directed routing.
- *Atlas Greedy DAG*: To account for complex, non-tree structures, we evaluate the Model Atlas framework [29]. This approach constructs a Directed Acyclic Graph (DAG) via a greedy algorithm, leveraging real-world structural priors such as temporal upload dynamics to determine edge directionality and explicitly accommodating multi-parent merged models.
- *Strict-Directional MDST*: We introduce a heavily constrained variant of the MDST. By setting the penalty for reverse edges (i.e., edges that violate the inferred directionality constraints) to infinity (∞), we strictly force the algorithm to adhere to the predicted parent-child hierarchical flow.
- *Undirected-MST*: We also implement a decoupled baseline. In the first stage, we construct a minimum spanning tree (MST) on the undirected graph using solely the distance matrix to determine topological connectivity. In the second stage, we subsequently assign directions to these established edges based on the independent directional information.

5 RQ1: Measurement Evaluation

Summary: L2+timestamp+xxgraph algorithm performs best.

6 RQ2: Layer Contribution Analysis

Summary: using QKV layer performs best. K layer contributes most.

7 RQ3: Failure Analysis

Multi-hop Analysis (RQ3): XX

8 RQ4: Integration Evaluation

9 Related Work

9.1 Model Identity Representation

Model watermarking deliberately embedded the triggers into the model to protect its intellectual property. Early mechanisms primarily targeted neural networks to defend against model extraction

Table 3: Lineage Detection Accuracy (%) w.r.t. Different Measurement Metrics, Direction Methods, and Graph Algorithms (RQ1)

		Macro-Accuracy										Micro-Accuracy									
Distances	**Dir.**	**L2**	**Cos.**	**AM**	**AMx**	**SA**	**SR**	**PDF**	**Ghost**	**Matrix**	**HuRef**	**L2**	**Cos.**	**AM**	**AMx**	**SA**	**SR**	**PDF**	**Ghost**	**Matrix**	**HuRef**
MoTHer	K	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
	T	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
Strict-Directional MDST	K	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
	T	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
Undirected-MST	K	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
	T	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
Atlas Greedy DAG	K	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
	T	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...

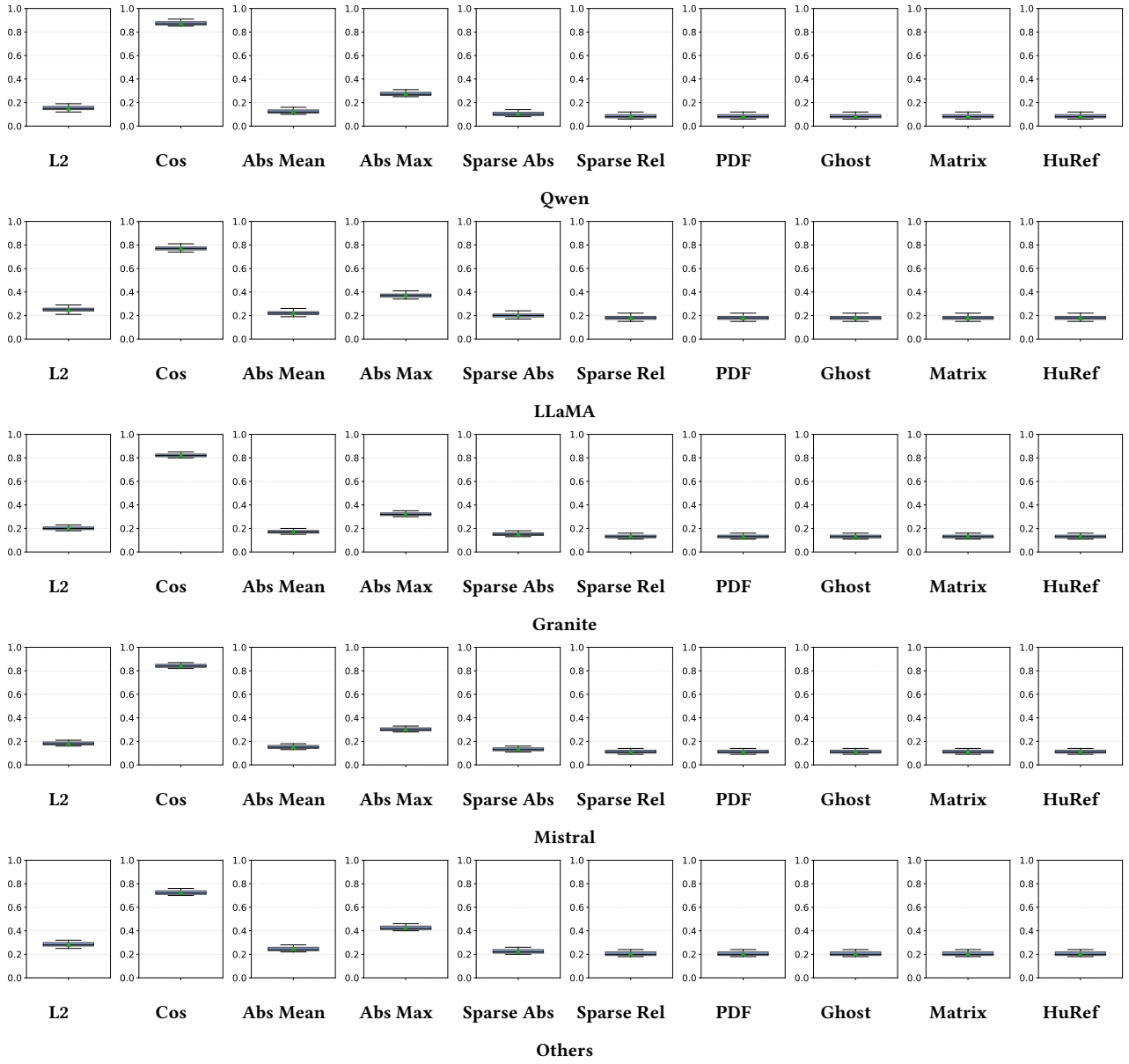


Figure 1: Distance metric distributions for different model families (RQ1).

and surrogate attacks [10, 23, 51, 52, 83, 106, 113]. Subsequent research has focused on embedding invisible and verifiable signatures into generative models [19, 33, 34, 54, 55, 84, 96, 115] to protect AI-generated multimedia content, including images [81], videos [33],

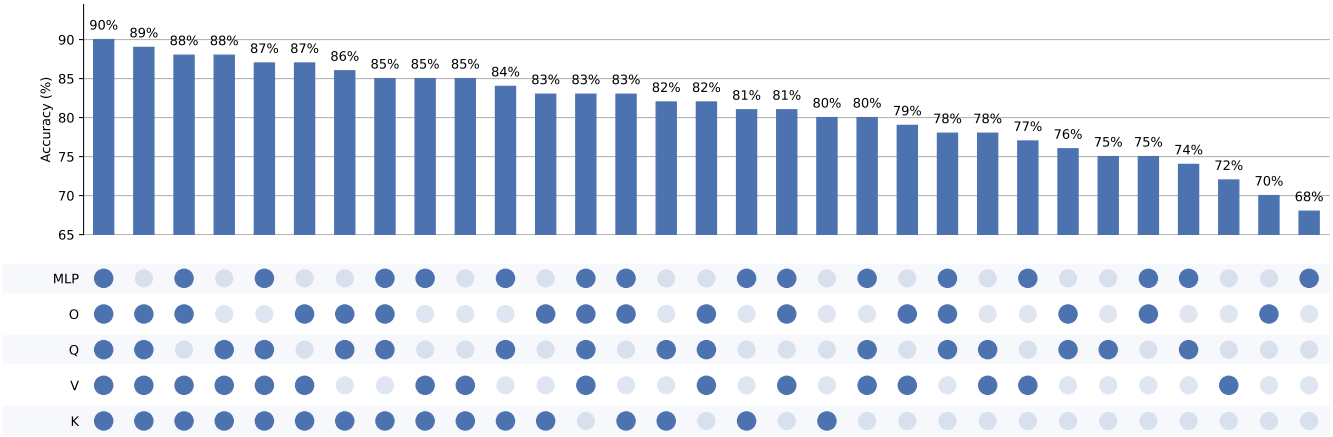


Figure 2: Accuracy of lineage detection with different combinations of transformer block components (RQ2).

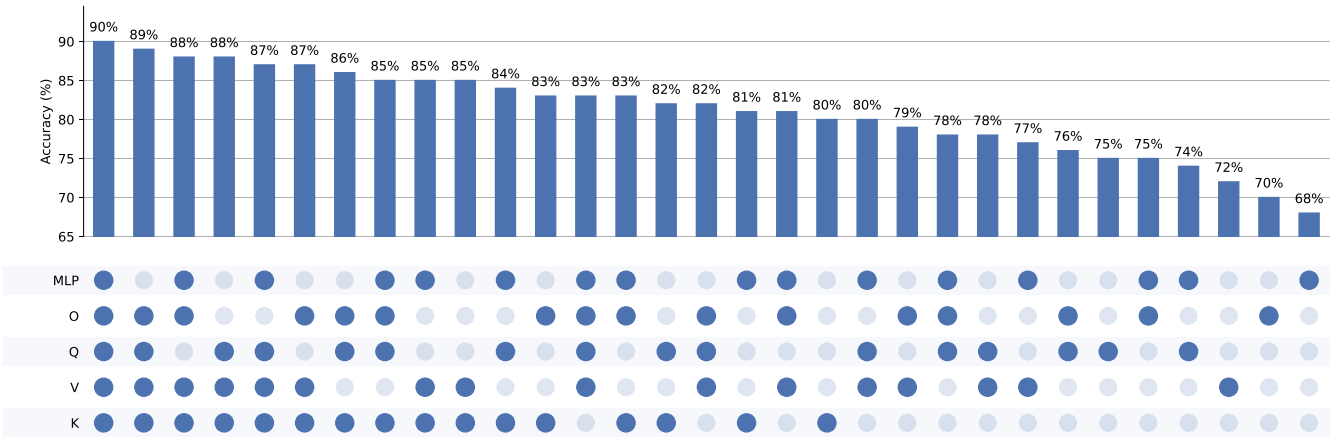


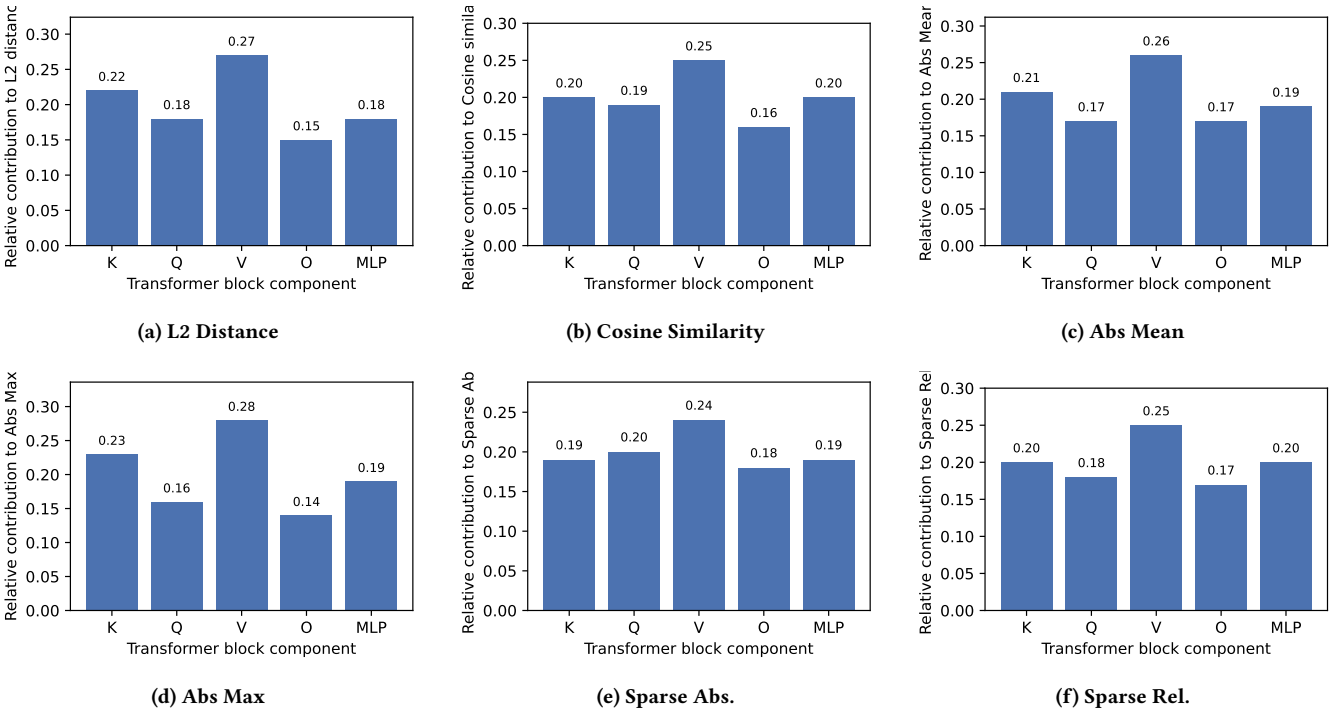
Figure 3: Accuracy2 of lineage detection with different combinations of transformer block components (RQ2).

and audio [40, 115]. Meanwhile, several studies aim to experiment watermark robustness against downstream fine-tuning [82] universal attacks [61], and forgery attacks [61]. Watermarking for LLMs was initially introduced to detect machine-generated text by biasing token selection during decoding [47]. Subsequent work has improved in terms of text quality, embedding capacity, and fine-grained detection capabilities [11, 20, 21, 31, 38, 41, 49, 56, 67, 69, 77, 86, 99, 111]. Meanwhile, adversarial studies have demonstrated evasion via paraphrasing, substitution, and cross-lingual transformations [28, 46, 80, 85]. In addition, recent approaches address challenges such as unforgeability [53] and support multi-user watermarking schemes [15]. Beyond model-level protection, emerging work extends watermarking techniques to data [64], text [112], code [74, 92], and prompts [97].

Model fingerprinting aims to build distinctive identifiers of models, encompassing both black-box behavioral profiling and white-box static analysis. Black-box fingerprinting relies on analyzing a model's dynamic inference behavior in response to carefully crafted inputs [26, 66, 95], including modeling models' outputs as unique vector subspaces [63, 65], functional representations [87], constructed prompts as adversarial suffixes [27, 90, 91]. White-box fingerprinting, in contrast, leverages access to internal model states to derive distinctive model profiles. Dynamic white-box approaches capture runtime characteristics, including feature representations [107], gradient tensors [88], activation differences [89], and token biases [76]. Static white-box fingerprinting captures intrinsic weight characteristics, including statistical distributions [100] (e.g., L_2 distances of model tensors [30]), directionally invariant terms [105], SVD-based spectral signatures [78], and matrix-driven homology proofs [108].

9.2 Model Relation Analysis

Building upon model fingerprinting techniques, recent research has advanced towards reconstructing the model relationships. Using dynamic white-box fingerprints, Wu et al. [88] proposes TensorGuard, which is a gradient-based fingerprinting to detect LLM lineage and classifies model families by analyzing layer-wise gradient responses to input perturbations. Besides, Oyama et al. [63] construct a model map using log-likelihood vectors from over 1,000 models. Wu et al. [87] introduces LLM DNA, a theoretically grounded, low-dimensional representation of model behavior. Using static



(a) L2 Distance (b) Cosine Similarity (c) Abs Mean

(d) Abs Max (e) Sparse Abs. (f) Sparse Rel.

Figure 4: Contribution of different transformer block components (K/Q/V/O/MLP) to various distance metrics (RQ2).

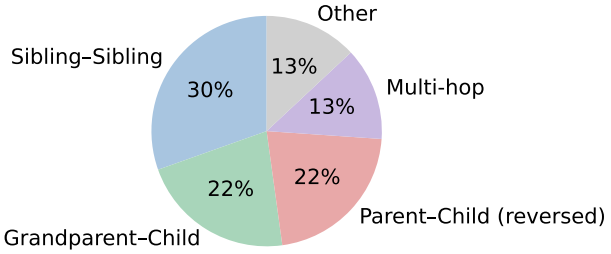


Figure 5: Proportion of different types of erroneous edges in the constructed lineage graph (RQ3).

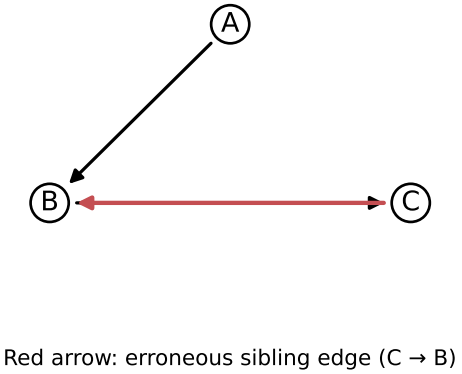


Figure 6: Case study illustrating erroneous edges in the constructed lineage graph (RQ3).

fingerprints, Zhang et al. [107] compares the pairwise feature representations of the two models using centered kernel alignment while remaining robust to model modifications like fine-tuning, pruning, and merging. Zhang et al. [108] proposes to detects and reconstructs weight correspondences using matrix analysis and statistical testing for LLM plagiarism detection without requiring inference. Based on fingerprinting and similarity analysis, recent research has begun to explicitly model LLM lineage. For instance, Yax et al.[98] apply phylogenetic algorithms to LLMs by leveraging output-based similarity to construct evolutionary trees that capture model relationships. Complementing this perspective, Yu et al.[101] propose neural lineage detection methods, including both learning-free and learning-based approaches, to identify parent and ancestral relationships among models. Shifting from behavioral to structural analysis, Horwitz [30] introduces MoTHer, which reconstructs lineage trees by analyzing neural network weights and

inferring parent-child relationships via a directed minimum spanning tree. They further proposes the Model Atlas [29], a unified graph-based framework that captures models, their attributes, and transformation relationships in more comprehensive view of the model ecosystem. We ...

Specifically regarding graph construction, Horwitz et al. [30] build a Model Tree by calculating pairwise weight distances to identify topological connections and utilizing weight kurtosis monotonicity to infer edge directions, ultimately recovering the structure via a minimum directed spanning tree (MDST) algorithm [14, 17]. Expanding this to a global scale, their Model Atlas framework [29] constructs a non-tree Directed Acyclic Graph (DAG) where nodes represent models and edges denote weight transformations. To chart unknown regions of this graph, they employ a greedy algorithm guided by real-world structural priors. This process involves leveraging temporal upload dynamics to determine edge directionality, treating quantized models as leaf nodes, and explicitly accommodating multi-parent merged models to map complex relationships across massive model repositories.

9.3 Software and LLM Supply Chain

Researchers have extensively studied the security and trust challenges in open-source software supply chains. Boughton [8] propose a framework for decomposing and measuring trust in open-source software supply chains by identifying factors that influence the trustworthiness of dependencies. From an adversarial perspective, Ladisa et al.[48] provide a systematic overview of attack techniques targeting open-source software supply chains. Complementary to these studies, Zahan et al. [103] evaluate security metrics and practices in open-source dependencies. Focusing on the JavaScript ecosystem, Zahan [104] analyze npm package metadata and propose weak-link signals to detect potential supply chain risks, while Miller [58] study how the npm ecosystem responds to dependency abandonment and its implications for supply chain sustainability. With the rapid growth of deep learning development, researchers have increasingly focused on the deep learning software supply chain. Tan et al.[75**?**] and Chen et al.[12] conduct exploratory studies on the supply chains of major deep learning frameworks (e.g., PyTorch and TensorFlow) to understand their downstream dependencies, development dynamics, collaboration patterns, and ecosystem differences. From another perspective, some studies focus on faults and failures in the deep learning stack. Huang et al.[36] perform an empirical investigation of dependency-related bugs in the deep learning software stack. Jajal et al.[39] examine interoperability issues of ONNX model converters in deep learning frameworks, highlighting the challenges of cross-framework model portability.

Software composition analysis is widely leveraged to systematically identify software components, supporting tasks such as vulnerability detection [43, 110], license analysis [24], and supply chain risk assessment [62]. The Software Bill of Materials (SBOM) [59, 70, 102] provides a structured description of the internal components of a software system and their dependency relationships. Traditional SCA techniques focus on extracting components from artifacts at different levels. For example, explicit dependencies can be parsed from build configuration files (e.g., package.json and pom.xml) [37, 114]. Static code analysis can be used to identify calls to third-party libraries [**?**]. Binary feature matching can detect repackaged components [93]. With the rise of LLMs, the composition analysis has expanded beyond traditional elements to include models, datasets, frameworks, prompts, and external tools invoked during execution [35, 79]. The concept of SBOM has been extended to the AI domain, such as AI Bill of Materials (AI BOM) [1, 7] and ML Bill of Materials (ML BOM) [4]. At the standardization level, CycloneDX has extended support for ML BOM, and SPDX 3.0 has also begun to discuss and support the modeling of AI-related assets [3]. However, achieving transparent composition analysis for LLM-based systems presents substantial technical challenges. Prior studies have found that such models may memorize training data and leak parts of the training corpus [9, 73]. To infer training data membership, membership inference attacks (MIA) [72] have been adopted. Mattern et al. [57] propose generating contrastive samples that are semantically similar to the target sample to amplify the sample differences. Zhang et al. [109] infer membership by measuring the deviation between the word frequency distribution within a document and that of the overall corpus. Shi et al.b [71] and Li et al. [50] propose the reference-free approaches based on the hypothesis that low-probability tokens tend to appear less frequently in member samples. Nevertheless, Duan et al. [16] found that the boundary between member and non-member samples is inherently fuzzy and continuous in the context of LLMs, leading to limited effectiveness. Besides, Ren et al. [68] studies the evolution of neural networks by constructing a large-scale neural network bill of materials database.

10 Discussion

References

[1] 2026. AI Bill Of Materials (AIBOM) Project. https://owaspaibom.org/. Accessed January 6, 2026.

[2] 2026. Hugging Face Models. https://huggingface.co/models?p=2&sort=trending. Accessed January 6, 2026.

[3] 2026. Implementing an AI BOM. https://spdx.dev/implementing-an-ai-bom/. Accessed January 6, 2026.

[4] 2026. Machine Learning Bill of Materials (ML-BOM). https://cyclonedx.org/capabilities/mlbom/. Accessed January 6, 2026.

[5] 2026. Model Cards. https://huggingface.co/docs/hub/model-cards. Accessed January 6, 2026.

[6] Ansh Arora, Xuanli He, Maximilian Mozes, Srinibas Swain, Mark Dras, and Qiongkai Xu. 2024. Here's a Free Lunch: Sanitizing Backdoored Models with Model Merge. In *Annual Meeting of the Association for Computational Linguistics*. https://api.semanticscholar.org/CorpusID:268091274

[7] Karen Bennet, Gopi Krishnan Rajbahadur, Arthit Suriyawongkul, and Kate Stewart. 2025. Implementing ai bill of materials (ai bom) with spdx 3.0: A comprehensive guide to creating ai and dataset bill of materials. *arXiv preprint arXiv:2504.16743* (2025).

[8] Lina Boughton, Courtney Miller, Yasemin Acar, Dominik Wermke, and Christian Kästner. 2024. Decomposing and measuring trust in open-source software supply chains. In *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results*. 57–61.

[9] Nicholas Carlini, Florian Tramer, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom Brown, Dawn Song, Ulfar Erlingsson, et al. 2021. Extracting training data from large language models. In *30th USENIX security symposium (USENIX Security 21)*. 2633–2650.

[10] Laurent Charette, Lingyang Chu, Yizhou Chen, Jian Pei, Lanjun Wang, and Yong Zhang. 2022. Cosine model watermarking against ensemble distillation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 36. 9512–9520.

[11] Ruibo Chen, Yihan Wu, Junfeng Guo, and Heng Huang. 2025. Improved unbiased watermark for large language models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 20587–20601.

[12] Yunqi Chen, Zhiyuan Wan, Yifei Zhuang, Ning Liu, David Lo, and Xiaohu Yang. 2025. Understanding the OSS communities of deep learning frameworks: a comparative case study of PyTorch and TensorFlow. *ACM Transactions on Software Engineering and Methodology* 34, 3 (2025), 1–30.

[13] Ziqian Chen, Zekai Chen, Susheng Wu, Bihuan Chen, Wenyan Song, Yiheng Huang, Zhuotong Zhou, Yiheng Cao, and Xin Peng. 2026. A First Look at Model Supply Chain: From the Risk Perspective. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE)*. IEEE.

[14] Yoeng-Jin Chu. 1965. On the shortest arborescence of a directed graph. *Scientia Sinica* 14 (1965), 1396–1400.

[15] Aloni Cohen, Alexander Hoover, and Gabe Schoenbach. 2025. Watermarking language models for many adaptive users. In *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2583–2601.

[16] Michael Duan, Anshuman Suri, Niloofar Mireshghallah, Sewon Min, Weijia Shi, Luke Zettlemoyer, Yulia Tsvetkov, Yejin Choi, David Evans, and Hannaneh Hajishirzi. 2024. Do membership inference attacks work on large language models? *arXiv preprint arXiv:2402.07841* (2024).

[17] Jack Edmonds et al. 1967. Optimum branchings. *Journal of Research of the national Bureau of Standards B* 71, 4 (1967), 233–240.

[18] Kazuki Egashira, Robin Staab, Mark Vero, Jingxuan He, and Martin Vechev. 2025. Mind the Gap: A Practical Attack on GGUF Quantization. *arXiv preprint arXiv:2505.23786* (2025).

[19] Pierre Fernandez, Guillaume Couairon, Hervé Jégou, Matthijs Douze, and Teddy Furon. 2023. The stable signature: Rooting watermarks in latent diffusion models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 22466–22477.

[20] Jiayi Fu, Xuandong Zhao, Ruihan Yang, Yuansen Zhang, Jiangjie Chen, and Yanghua Xiao. 2024. Gumbelsoft: Diversified language model watermarking via the gumbelmax-trick. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 5791–5808.

[21] Yu Fu, Deyi Xiong, and Yue Dong. 2024. Watermarking conditional text generation for ai detection: Unveiling challenges and a semantic-aware watermark remedy. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 18003–18011.

[22] Zihao Fu, Haoran Yang, Anthony Man-Cho So, Wai Lam, Lidong Bing, and Nigel Collier. 2022. On the Effectiveness of Parameter-Efficient Fine-Tuning. *ArXiv* abs/2211.15583 (2022). https://api.semanticscholar.org/CorpusID:254043494

[23] Guanhao Gan, Yiming Li, Dongxian Wu, and Shu-Tao Xia. 2023. Towards robust model watermark via reducing parametric vulnerability. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 4751–4761.

[24] Daniel M German and Ahmed E Hassan. 2009. License integration patterns: Addressing license mismatches in component-based development. In *2009 IEEE 31st international conference on software engineering*. IEEE, 188–198.

[25] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM workshop on artificial intelligence and security*. 79–90.

[26] Jiyang Guan, Jian Liang, and Ran He. 2022. Are you stealing my model? sample correlation for fingerprinting deep neural networks. *Advances in Neural Information Processing Systems* 35 (2022), 36571–36584.

[27] Martin Gubri, Dennis Ulmer, Hwaran Lee, Sangdoo Yun, and Seong Joon Oh. 2024. Trap: Targeted random adversarial prompt honeypot for black-box identification. In *Findings of the Association for Computational Linguistics: ACL 2024*. 11496–11517.

[28] Zhiwei He, Binglin Zhou, Hongkun Hao, Aiwei Liu, Xing Wang, Zhaopeng Tu, Zhuosheng Zhang, and Rui Wang. 2024. Can watermarks survive translation? on the cross-lingual consistency of text watermark for large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 4115–4129.

[29] Eliahu Horwitz, Nitzan Kurer, Jonathan Kahana, Liel Amar, and Yedid Hoshen. 2025. We Should Chart an Atlas of All the World's Models. *arXiv preprint arXiv:2503.10633* (2025).

[30] Eliahu Horwitz, Asaf Shul, and Yedid Hoshen. 2024. Unsupervised model tree heritage recovery. *arXiv preprint arXiv:2405.18432* (2024).

[31] Abe Hou, Jingyu Zhang, Yichen Wang, Daniel Khashabi, and Tianxing He. 2024. k-SemStamp: A clustering-based semantic watermark for detection of machine-generated text. In *Findings of the Association for Computational Linguistics: ACL 2024*. 1706–1715.

[32] Qiang Hu, Xiaofei Xie, Sen Chen, Lili Quan, and Lei Ma. 2025. Large language model supply chain: Open problems from the security perspective. In *Proceedings of the 34th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 169–173.

[33] Runyi Hu, Jie Zhang, Yiming Li, Jiwei Li, Qing Guo, Han Qiu, and Tianwei Zhang. 2025. Videoshield: Regulating diffusion-based video generation models via watermarking. *arXiv preprint arXiv:2501.14195* (2025).

[34] Huayang Huang, Yu Wu, and Qian Wang. 2024. Robin: Robust and invisible watermarks for diffusion models with adversarial optimization. *Advances in Neural Information Processing Systems* 37 (2024), 3937–3963.

[35] Kaifeng Huang, Bihuan Chen, You Lu, Susheng Wu, Dingji Wang, Yiheng Huang, Haowen Jiang, Zhuotong Zhou, Junming Cao, and Xin Peng. 2024. Lifting the Veil on Composition, Risks, and Mitigations of the Large Language Model Supply Chain. *arXiv preprint arXiv:2410.21218* (2024).

[36] Kaifeng Huang, Bihuan Chen, Susheng Wu, Junming Cao, Lei Ma, and Xin Peng. 2023. Demystifying dependency bugs in deep learning stack. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 450–462.

[37] Kaifeng Huang, Bihuan Chen, Congying Xu, Ying Wang, Bowen Shi, Xin Peng, Yijian Wu, and Yang Liu. 2022. Characterizing usages, updates and risks of third-party libraries in Java projects. *Empirical Software Engineering* 27, 4 (2022), 90.

[38] Mingjia Huo, Sai Ashish Somayajula, Youwei Liang, Ruisi Zhang, Farinaz Koushanfar, and Pengtao Xie. 2024. Token-specific watermarking with enhanced detectability and semantic coherence for large language models. *arXiv preprint arXiv:2402.18059* (2024).

[39] Purvish Jajal, Wenxin Jiang, Arav Tewari, Erik Kocinare, Joseph Woo, Anusha Sarraf, Yung-Hsiang Lu, George K Thiruvathukal, and James C Davis. 2024. Interoperability in deep learning: A user survey and failure analysis of onnx model converters. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1466–1478.

[40] Shengpeng Ji, Ziyue Jiang, Jialong Zuo, Minghui Fang, Yifu Chen, Tao Jin, and Zhou Zhao. 2025. Speech watermarking with discrete intermediate representations. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 24239–24247.

[41] Zhe Ji, Qiansiqi Hu, Yicheng Zheng, Liyao Xiang, and Xinbing Wang. 2024. A principled approach to natural language watermarking. In *Proceedings of the 32nd ACM International Conference on Multimedia*. 2908–2916.

[42] Jinyuan Jia, Yupei Liu, and Neil Zhenqiang Gong. 2022. Badencoder: Backdoor attacks to pre-trained encoders in self-supervised learning. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2043–2059.

[43] Ling Jiang, Junwen An, Huihui Huang, Qiyi Tang, Sen Nie, Shi Wu, and Yuqun Zhang. 2024. Binaryai: Binary software composition analysis via intelligent binary source code matching. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.

[44] Wenxin Jiang, Nicholas Synovic, Matt Hyatt, Taylor R Schorlemmer, Rohan Sethi, Yung-Hsiang Lu, George K Thiruvathukal, and James C Davis. 2023. An empirical study of pre-trained model reuse in the hugging face deep learning model registry. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2463–2475.

[45] Wenxin Jiang, Nicholas M. Synovic, Matt Hyatt, Taylor R. Schorlemmer, Rohan Sethi, Yung-Hsiang Lu, George K. Thiruvathukal, and James C. Davis. 2023. An Empirical Study of Pre-Trained Model Reuse in the Hugging Face Deep Learning Model Registry. *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)* (2023), 2463–2475. https://api.semanticscholar.org/CorpusID:257365660

[46] Nikola Jovanović, Robin Staab, and Martin Vechev. 2024. Watermark stealing in large language models. *arXiv preprint arXiv:2402.19361* (2024).

[47] John Kirchenbauer, Jonas Geiping, Yuxin Wen, Jonathan Katz, Ian Miers, and Tom Goldstein. 2023. A watermark for large language models. In *International conference on machine learning*. PMLR, 17061–17084.

[48] Piergiorgio Ladisa, Henrik Plate, Matias Martinez, and Olivier Barais. 2023. Sok: Taxonomy of attacks on open-source software supply chains. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1509–1526.

[49] Xingchi Li, Guanxun Li, and Xianyang Zhang. 2024. Segmenting watermarked texts from language models. *Advances in Neural Information Processing Systems* 37 (2024), 14634–14665.

[50] Yuanheng Li, Zhuoyang Chen, Xiaoyun Liu, Yuhao Wang, Mingwei Liu, Yang Shi, Kaifeng Huang, and Shengjie Zhao. 2025. Uncovering Pretraining Code in LLMs: A Syntax-Aware Attribution Approach. *The 40th Annual AAAI Conference on Artificial Intelligence* (2025).

[51] Yiming Li, Linghui Zhu, Xiaojun Jia, Yong Jiang, Shu-Tao Xia, and Xiaochun Cao. 2022. Defending against model stealing via verifying embedded external features. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 36. 1464–1472.

[52] Yiming Li, Mingyan Zhu, Xue Yang, Yong Jiang, Tao Wei, and Shu-Tao Xia. 2023. Black-box dataset ownership verification via backdoor watermarking. *IEEE Transactions on Information Forensics and Security* 18 (2023), 2318–2332.

[53] Aiwei Liu, Leyi Pan, Xuming Hu, Shu'ang Li, Lijie Wen, Irwin King, and Philip S Yu. 2023. An unforgeable publicly verifiable watermark for large language models. *arXiv preprint arXiv:2307.16230* (2023).

[54] Guan-Horng Liu, Tianrong Chen, Evangelos Theodorou, and Molei Tao. 2023. Mirror diffusion models for constrained and watermarked generation. *Advances in Neural Information Processing Systems* 36 (2023), 42898–42917.

[55] Weizhi Liu, Yue Li, Dongdong Lin, Hui Tian, and Haizhou Li. 2024. Groot: Generating robust watermark for diffusion-model-based audio synthesis. In *Proceedings of the 32nd ACM International Conference on Multimedia*. 3294–3302.

[56] Yepeng Liu and Yuheng Bu. 2024. Adaptive text watermark for large language models. *arXiv preprint arXiv:2401.13927* (2024).

[57] Justus Mattern, Fatemehsadat Mireshghallah, Zhijing Jin, Bernhard Schölkopf, Mrinmaya Sachan, and Taylor Berg-Kirkpatrick. 2023. Membership inference attacks against language models via neighbourhood comparison. In *Findings of the Association for Computational Linguistics: ACL 2023*. 11330–11343.

[58] Courtney Miller, Mahmoud Jahanshahi, Audris Mockus, Bogdan Vasilescu, and Christian Kastner. 2025. Understanding the response to open-source dependency abandonment in the npm ecosystem. (2025).

[59] Mehdi Mirakhorli, Derek Garcia, Schuyler Dillon, Kevin Laporte, Matthew Morrison, Henry Lu, Viktoria Koscinski, and Christopher Enoch. 2024. A landscape study of open source and proprietary tools for software bill of materials (sbom). *arXiv preprint arXiv:2402.11151* (2024).

[60] Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. 2018. Model Cards for Model Reporting. *Proceedings of the Conference on Fairness, Accountability, and Transparency* (2018). https://api.semanticscholar.org/CorpusID:52946140

[61] Andreas Müller, Denis Lukovnikov, Jonas Thietke, Asja Fischer, and Erwin Quiring. 2025. Black-box forgery attacks on semantic watermarks for diffusion models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 20937–20946.

[62] Sabato Nocera, Sira Vegas, Giuseppe Scanniello, and Natalia Juristo Juzgado. 2025. Software Composition Analysis and Supply Chain Security in Apache Projects: an Empirical Study. *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)* (2025), 103–115.

[63] Momose Oyama, Hiroaki Yamagiwa, Yusuke Takase, and Hidetoshi Shimodaira. 2025. Mapping 1,000+ Language Models via the Log-Likelihood Vector. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 32983–33038.

[64] Leyi Pan, Aiwei Liu, Shiyu Huang, Yijian Lu, Xuming Hu, Lijie Wen, Irwin King, and Philip S Yu. 2025. Can LLM Watermarks Robustly Prevent Unauthorized Knowledge Distillation?. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 13228–13251.

[65] Dario Pasquini, Evgenios M Kornaropoulos, and Giuseppe Ateniese. 2025. {LLMmap}: Fingerprinting for large language models. In *34th USENIX Security Symposium (USENIX Security 25)*. 299–318.

[66] Zirui Peng, Shaofeng Li, Guoxing Chen, Cheng Zhang, Haojin Zhu, and Minhui Xue. 2022. Fingerprinting deep neural networks globally via universal adversarial perturbations. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 13430–13439.

[67] Jipeng Qiang, Shiyu Zhu, Yun Li, Yi Zhu, Yunhao Yuan, and Xindong Wu. 2023. Natural language watermarking via paraphraser-based lexical substitution. *Artificial Intelligence* 317 (2023), 103859.

[68] Xiaoning Ren, Yuhang Ye, Xiongfei Wu, Yueming Wu, and Yinxing Xue. 2025. Demystifying the Evolution of Neural Networks with BOM Analysis: Insights from a Large-Scale Study of 55,997 GitHub Repositories. *Proceddings of the 40th IEEE/ACM International Conference on Automated Software Engineering* (2025).

[69] Yubing Ren, Ping Guo, Yanan Cao, and Wei Ma. 2024. Subtle signatures, strong shields: Advancing robust and imperceptible watermarking in large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*. 5508–5519.

[70] Vandana Verma Sehgal and PS Ambili. 2023. A taxonomy and survey of software bill of materials (SBOM) generation approaches. In *Analytics Global Conference*. 40–51.

[71] Weijia Shi, Anirudh Ajith, Mengzhou Xia, Yangsibo Huang, Daogao Liu, Terra Blevins, Danqi Chen, and Luke Zettlemoyer. 2023. Detecting pretraining data from large language models. *arXiv preprint arXiv:2310.16789* (2023).

[72] Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. 2017. Membership inference attacks against machine learning models. In *2017 IEEE symposium on security and privacy (SP)*. IEEE, 3–18.

[73] Congzheng Song and Vitaly Shmatikov. 2019. Auditing data provenance in text-generation models. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 196–206.

[74] Zhensu Sun, Xiaoning Du, Fu Song, and Li Li. 2023. Codemark: Imperceptible watermarking for code datasets against neural code completion models. In *Proceedings of the 31st ACM joint European software engineering conference and symposium on the foundations of software engineering*. 1561–1572.

[75] Xin Tan, Kai Gao, Minghui Zhou, and Li Zhang. 2022. An exploratory study of deep learning supply chain. In *Proceedings of the 44th international conference on software engineering*. 86–98.

[76] Yao Tong, Haonan Wang, Siquan Li, Kenji Kawaguchi, and Tianyang Hu. 2025. SeedPrints: Fingerprints Can Even Tell Which Seed Your Large Language Model Was Trained From. *arXiv preprint arXiv:2509.26404* (2025).

[77] Lean Wang, Wenkai Yang, Deli Chen, Hao Zhou, Yankai Lin, Fandong Meng, Jie Zhou, and Xu Sun. 2023. Towards codable watermarking for injecting multi-bits information to LLMs. *arXiv preprint arXiv:2307.15992* (2023).

[78] Suqing Wang, Ziyang Ma, Li Xinyi, and Zuchao Li. 2025. Ghost in the Transformer: Detecting Model Reuse with Invariant Spectral Signatures. *arXiv preprint arXiv:2511.06390* (2025).

[79] Shenao Wang, Yanjie Zhao, Xinyi Hou, and Haoyu Wang. 2025. Large language model supply chain: A research agenda. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 1–46.

[80] Yichen Wang, Shangbin Feng, Abe Hou, Xiao Pu, Chao Shen, Xiaoming Liu, Yulia Tsvetkov, and Tianxing He. 2024. Stumbling blocks: Stress testing the robustness of machine-generated text detectors under attacks. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2894–2925.

[81] Yuchen Wang, Xingyu Zhu, Guanhui Ye, Shiyao Zhang, and Xuetao Wei. 2024. Achieving resolution-agnostic dnn-based image watermarking: A novel perspective of implicit neural representation. In *Proceedings of the 32nd ACM International Conference on Multimedia*. 10354–10362.

[82] Zilan Wang, Junfeng Guo, Jiacheng Zhu, Yiming Li, Heng Huang, Muhao Chen, and Zhengzhong Tu. 2025. Sleepermark: Towards robust watermark against fine-tuning text-to-image diffusion models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 8213–8224.

[83] Zhenyi Wang, Yihan Wu, and Heng Huang. 2024. Defense against model extraction attack by bayesian active watermarking. In *Forty-first International Conference on Machine Learning*.

[84] Yuxin Wen, John Kirchenbauer, Jonas Geiping, and Tom Goldstein. 2023. Tree-rings watermarks: Invisible fingerprints for diffusion images. *Advances in Neural Information Processing Systems* 36 (2023), 58047–58063.

[85] Qilong Wu and Varun Chandrasekaran. 2024. Bypassing llm watermarks with color-aware substitutions. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 8549–8581.

[86] Yihan Wu, Zhengmian Hu, Junfeng Guo, Hongyang Zhang, and Heng Huang. 2023. A resilient and accessible distribution-preserving watermark for large language models. *arXiv preprint arXiv:2310.07710* (2023).

[87] Zhaomin Wu, Haodong Zhao, Ziyang Wang, Jizhou Guo, Qian Wang, and Bingsheng He. 2025. LLM DNA: Tracing Model Evolution via Functional Representations. *arXiv preprint arXiv:2509.24496* (2025).

[88] Zehao Wu, Yanjie Zhao, and Haoyu Wang. 2025. TensorGuard: Gradient-Based Model Fingerprinting for LLM Similarity Detection and Family Classification. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 445–456.

[89] Zhiqiu Xu, Amish Sethi, Mayur Naik, and Ser-Nam Lim. 2025. Delta Activations: A Representation for Finetuned Large Language Models. *arXiv preprint arXiv:2509.04442* (2025).

[90] Zhenhua Xu, Yiran Zhao, Mengting Zhong, Dezhang Kong, Changting Lin, Tong Qiao, and Meng Han. 2026. DNF: Dual-Layer Nested Fingerprinting for Large Language Model Intellectual Property Protection. *arXiv preprint arXiv:2601.08223* (2026).

[91] Shojiro Yamabe, Futa Kai Waseda, Tsubasa Takahashi, and Koki Wataoka. 2025. MergePrint: Merge-Resistant Fingerprints for Robust Black-box Ownership Verification of Large Language Models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 6894–6916.

[92] Borui Yang, Wei Li, Liyao Xiang, and Bo Li. 2024. Srcmarker: Dual-channel source code watermarking via scalable code transformations. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 4088–4106.

[93] Can Yang, Zhengzi Xu, Hongxu Chen, Yang Liu, Xiaorui Gong, and Baoxu Liu. 2022. Modx: binary level partially imported third-party library detection via program modularization and semantic matching. In *Proceedings of the 44th International Conference on Software Engineering*. 1393–1405.

[94] Jiawei Yang, Zhongbo Li, Zeqin Feng, and Yongqiang Xie. 2025. A Survey On Neural Network Quantization. *Proceedings of the 2025 6th International Conference on Computer Information and Big Data Applications* (2025). https://api.semanticscholar.org/CorpusID:281040689

[95] Kang Yang, Run Wang, and Lina Wang. 2022. MetaFinger: Fingerprinting the Deep Neural Networks with Meta-training.. In *IJCAI*. 776–782.

[96] Zijin Yang, Kai Zeng, Kejiang Chen, Han Fang, Weiming Zhang, and Nenghai Yu. 2024. Gaussian shading: Provable performance-lossless image watermarking for diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 12162–12171.

[97] Hongwei Yao, Jian Lou, Zhan Qin, and Kui Ren. 2024. Promptcare: Prompt copyright protection by watermark injection and verification. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 845–861.

[98] Nicolas Yax, Pierre-Yves Oudeyer, and Stefano Palminteri. 2024. PhyloLM: Inferring the phylogeny of large language models and predicting their performances in benchmarks. *arXiv preprint arXiv:2404.04671* (2024).

[99] KiYoon Yoo, Wonhyuk Ahn, Jiho Jang, and Nojun Kwak. 2023. Robust multi-bit natural language watermarking through invariant features. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2092–2115.

[100] Do-hyeon Yoon, Minsoo Chun, Thomas Allen, Hans Müller, Min Wang, and Rajesh Sharma. 2025. Intrinsic Fingerprint of LLMs: Continue Training is NOT All You Need to Steal A Model! *arXiv preprint arXiv:2507.03014* (2025).

[101] Runpeng Yu and Xinchao Wang. 2024. Neural lineage. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 4797–4807.

[102] Sheng Yu, Wei Song, Xunchao Hu, and Heng Yin. 2024. On the correctness of metadata-based SBOM generation: A differential analysis approach. In *2024 54th Annual IEEE/IFIP International conference on dependable systems and networks (DSN)*. 29–36.

[103] Nusrat Zahan. 2023. Software supply chain risk assessment framework. In *2023 IEEE/ACM 45th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. IEEE, 251–255.

[104] Nusrat Zahan, Thomas Zimmermann, Patrice Godefroid, Brendan Murphy, Chandra Maddila, and Laurie Williams. 2022. What are weak links in the npm supply chain?. In *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice*. 331–340.

[105] Boyi Zeng, Lizheng Wang, Yuncong Hu, Yi Xu, Chenghu Zhou, Xinbing Wang, Yu Yu, and Zhouhan Lin. 2024. Huref: Human-readable fingerprint for large language models. *Advances in Neural Information Processing Systems* 37 (2024), 126332–126362.

[106] Jie Zhang, Dongdong Chen, Jing Liao, Zehua Ma, Han Fang, Weiming Zhang, Huamin Feng, Gang Hua, and Nenghai Yu. 2024. Robust model watermarking for image processing networks via structure consistency. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 46, 10 (2024), 6985–6992.

[107] Jie Zhang, Dongrui Liu, Chen Qian, Linfeng Zhang, Yong Liu, Yu Qiao, and Jing Shao. 2024. Reef: Representation encoding fingerprints for large language models. *arXiv preprint arXiv:2410.14273* (2024).

[108] Ruichong Zhang. 2025. Matrix-driven instant review: Confident detection and reconstruction of llm plagiarism on pc. *arXiv e-prints* (2025), arXiv–2508.

[109] Weichao Zhang, Ruqing Zhang, Jiafeng Guo, Maarten de Rijke, Yixing Fan, and Xueqi Cheng. 2024. Pretraining data detection for large language models: A divergence-based calibration method. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 5263–5274.

[110] Lida Zhao, Sen Chen, Zhengzi Xu, Chengwei Liu, Lyuye Zhang, Jiahui Wu, Jun Sun, and Yang Liu. 2023. Software composition analysis for vulnerability detection: An empirical study on Java projects. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 960–972.

[111] Xuandong Zhao, Chenwen Liao, Yu-Xiang Wang, and Lei Li. 2025. Efficiently identifying watermarked segments in mixed-source texts. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 6304–6316.

[112] Tong Zhou, Xuandong Zhao, Xiaolin Xu, and Shaolei Ren. 2024. Bileve: Securing text provenance in large language models against spoofing with bi-level signature. *Advances in Neural Information Processing Systems* 37 (2024), 56054–56075.

[113] Hongyu Zhu, Sichu Liang, Wentao Hu, Li Fangqi, Ju Jia, and Shi-Lin Wang. 2024. Reliable model watermarking: Defending against theft without compromising on evasion. In *Proceedings of the 32nd ACM International Conference on Multimedia*. 10124–10133.

[114] Markus Zimmermann, Cristian-Alexandru Staicu, Cam Tenny, and Michael Pradel. 2019. Small world with high risks: A study of security threats in the npm ecosystem. In *28th USENIX Security symposium (USENIX security 19)*. 995–1010.

[115] Wei Zong, Yang-Wai Chow, Willy Susilo, Joonsang Baek, and Seyit Camtepe. 2025. {AudioMarkNet}: Audio watermarking for deepfake speech detection. In *34th USENIX Security Symposium (USENIX Security 25)*. 4663–4682.